

FlowState -RL

Dynamic Difficulty Adaptation using Reinforcement Learning
Based on Csikszentmihalyi's Flow Theory

- PPO Algorithm
- Gymnasium Env
- 6D State Space
- 9 Discrete Actions
- 2 Game Environments

An RL-powered Game Director that acts as an invisible adaptive system, continuously monitoring real-time player behavior and dynamically tuning game difficulty parameters to keep players in the optimal psychological engagement zone — the Flow Channel.

Not an AI playing a game. An AI shaping the game around a human.

TABLE OF CONTENTS

1. Problem Statement	2
2. System Architecture	3
3. RL Formulation	4
4. PPO Agent Design	5
5. Libraries & Environment	6
6. Components to Develop	7
7. Development Timeline	8
8. Dashboard Design	8

2025

1. Problem Statement

01 · PROBLEM STATEMENT

In modern video games, static difficulty settings create a fundamental engagement problem. Developers choose a fixed challenge level that inevitably mismatches with the full spectrum of player skill. This mismatch manifests as two distinct failure modes that destroy long-term player retention and game enjoyment.

FAILURE MODE 1	FAILURE MODE 2
Skilled Player Gets Bored Challenge is too low. The player destroys enemies with no effort, score climbs automatically, and there is no tension or engagement. Session ends within minutes.	Beginner Gets Frustrated Challenge is too high. The player dies repeatedly, cannot progress, skill gap is insurmountable. Game feels unfair. Player quits in frustration.

The Flow Theory Foundation

Mihaly Csikszentmihalyi's Flow Theory (1990) identifies the psychological state of "flow" as optimal human experience — a state of deep focus and effortless engagement occurring when challenge precisely matches skill level. This is the scientific basis for FlowState-RL's reward function and objective.

ANXIETY ZONE	FLOW CHANNEL	BOREDOM ZONE
Challenge overwhelmed Skill Constant Player deaths Quits in frustration	Challenge = Skill Peak engagement Optimal performance Time distortion effect	Challenge << Skill Player under-stimulated No meaningful decisions Quits from tedium

The Solution: FlowState-RL

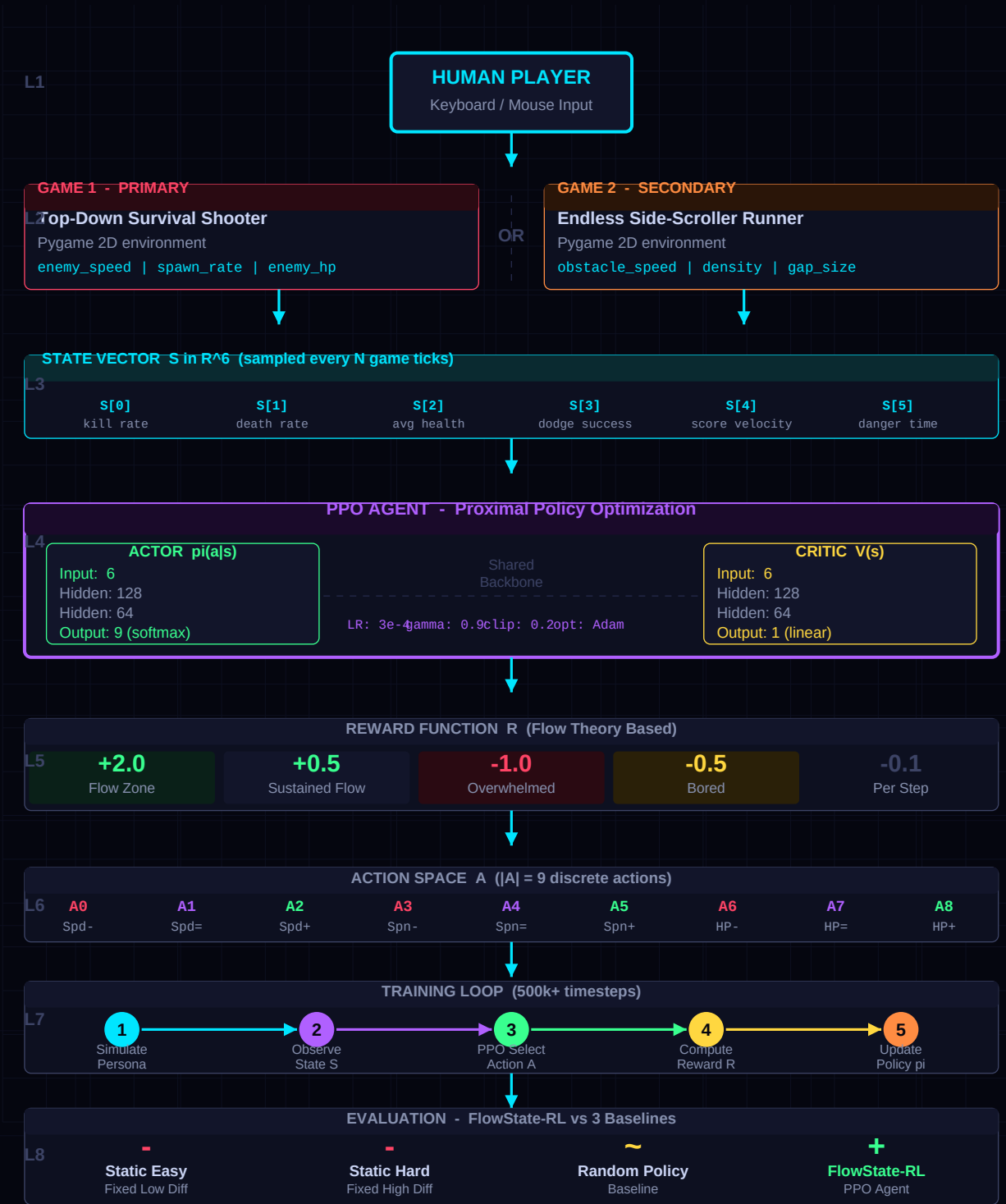
FlowState-RL trains a PPO reinforcement learning agent to act as an invisible **Game Director**. It continuously observes 6 real-time player behavioral metrics, selects from 9 game parameter adjustments, and receives rewards based on how close the player is to the Flow Zone. The agent learns to keep any player — beginner, expert, or anything in between — in a state of optimal engagement by dynamically reshaping the game around them.

This is **not** an AI playing a game. This is an AI shaping the game around a human.

2. System Architecture

02 · SYSTEM ARCHITECTURE DIAGRAM

The complete FlowState-RL system operates as 8 connected layers, from the human player input through the game environments, state observation, PPO agent decision-making, reward computation, action execution, training loop, and final evaluation comparison.



3. RL Formulation

03 · RL FORMULATION

3.1 Environment

A custom 2D top-down survival shooter built with Pygame, where enemies spawn and chase the player. The RL agent controls the game parameters, not the player. A second environment (endless runner) is developed for transfer learning experiments.

3.2 State Space S

Index	Feature	Description	Range	Captures
S[0]	kill_rate	Enemies killed per second (last 10s)	[0, inf)	Offensive skill
S[1]	death_rate	Player deaths per minute	[0, inf)	Survivability
S[2]	avg_health	Average HP% over last episode	[0, 1]	Resource management
S[3]	dodge_success	% of attacks successfully dodged	[0, 1]	Defensive skill
S[4]	score_velocity	Rate of score change over time	R	Progress rate
S[5]	danger_time	% of time player HP < 30%	[0, 1]	Risk exposure

3.3 Action Space A

ID	Action	Parameter	Delta	Range	ID	Action	Parameter	Delta	Range
A0	Decrease	enemy_speed	-0.2	[1.0, 5.0]	A5	Increase	spawn_rate	+0.2	[0.5, 3.0]
A1	Keep	enemy_speed	0	[1.0, 5.0]	A6	Decrease	enemy_hp	-0.5	[1.0, 5.0]
A2	Increase	enemy_speed	+0.2	[1.0, 5.0]	A7	Keep	enemy_hp	0	[1.0, 5.0]
A3	Decrease	spawn_rate	-0.2	[0.5, 3.0]	A8	Increase	enemy_hp	+0.5	[1.0, 5.0]
A4	Keep	spawn_rate	0	[0.5, 3.0]					

3.4 Reward Function R

Condition	Reward	Trigger	Psychological Meaning
In Flow Zone	+2.0	performance in [0.5, 0.7]	Optimal challenge-skill balance
Sustained Flow Bonus	+0.5	flow_streak >= 30 steps	Maintained engagement over time
Overwhelmed	-1.0	death_rate > 0.7	Game is too hard, anxiety state

Bored	-0.5	kill_rate>0.8 and death<0.1	Game is too easy, no engagement
Outside Flow (per step)	-0.1	All other states	Encourages faster correction

Flow Score = 1.0 - |normalized_performance - 0.6| | Performance = 0.3*kill + 0.25*health + 0.25*dodge + 0.2*score_vel - 0.3*death - 0.2*danger

4. PPO Agent Design

04 · PPO AGENT DESIGN

4.1 Algorithm Selection: Why PPO?

Criterion	PPO Advantage	Alternatives Considered
Policy Stability	Clipped objective (eps=0.2) prevents catastrophic policy collapse during live game adaptation	DQN: no clipping, unstable for continuous adaptation
Sample Efficiency	Multiple epochs per rollout; efficient for small 6D state spaces	REINFORCE: single update per episode, wasteful
Architecture Fit	Actor-Critic naturally maps to "how is player doing" vs "what to change"	Q-learning: value-only, no policy head
Real-Time Suitable	Converges within 500k steps; runs inference in microseconds	SAC: overkill for discrete 9-action space
Library Support	stable-baselines3 PPO: production-grade, well-tested, minimal boilerplate	Custom impl: error-prone, time-consuming

4.2 Network Architecture

Layer	Type	Size	Activation	Output	Purpose
Input	Dense	6	—	6 floats	State vector S
Hidden 1	Dense	128	ReLU	128	Feature extraction
Hidden 2	Dense	64	ReLU	64	Policy refinement
Actor Head	Dense	9	Softmax	pi(a s)	Action probabilities
Critic Head	Dense	1	Linear	V(s)	State value estimate

4.3 Hyperparameters

Parameter	Value	Parameter	Value	Parameter	Value
Learning Rate	3e-4	Discount gamma	0.99	Clip Range eps	0.2
N Steps	2048	Batch Size	64	N Epochs	10
Entropy Coef	0.01	Optimizer	Adam	Total Timesteps	500,000
Hidden Layers	[128, 64]	Policy	MlpPolicy	Eval Episodes	10

4.4 Player Personas (Training Data)

Since no real game is available initially, 4 simulated player personas generate realistic behavioral state vectors. Each persona has distinct statistical characteristics that stress-test different aspects of the reward function.

Persona	kill_rate	death_rate	avg_health	dodge_success	Behavior Description
Beginner	0.15 +/-0.08	0.75 +/-0.10	0.25 +/-0.08	0.15 +/-0.08	Struggles with the game. Dies often, rarely dodges, low kill rate.
Average	0.50 +/-0.10	0.40 +/-0.10	0.55 +/-0.08	0.45 +/-0.10	Moderate player with some deaths and decent dodging.
Expert	0.85 +/-0.07	0.08 +/-0.05	0.85 +/-0.07	0.82 +/-0.07	Dominates the game. Rarely dies, high dodge rate, fast killer.
Erratic	Switches	Switches	Switches	Switches	Randomly switches between Expert and Beginner stats every 20 steps.

5. Libraries & Environment Setup

05 · LIBRARIES & ENVIRONMENT SETUP

5.1 Core RL & Agent Libraries

Library	Version	Role in Project	Install
stable-baselines3	>=2.0	PPO implementation, training loop, model save/load, evaluation utilities	<code>pip install stable-baselines3</code>
gymnasium	>=0.29	Custom Env base class, observation/action spaces, Gym-compatible wrappers	<code>pip install gymnasium</code>
numpy	>=1.24	State vector construction, random noise generation, array operations	<code>pip install numpy</code>
torch	>=2.0	Neural network backend for PPO (auto-installed with stable-baselines3)	auto-installed
tensorboard	>=2.13	Live training visualization: reward curves, loss plots, policy entropy	<code>pip install tensorboard</code>

5.2 Game Environment Libraries

Library	Version	Role	Install
pygame	>=2.5	2D game rendering, player/enemy sprites, collision detection, game loop	<code>pip install pygame</code>
matplotlib	>=3.7	Training reward curves, flow zone %, evaluation comparison plots	<code>pip install matplotlib</code>
pandas	>=2.0	Logging training metrics to CSV, data manipulation for dashboard	<code>pip install pandas</code>

5.3 Dashboard Libraries

Library	Version	Role	Install
flask	>=3.0	Python API server to stream training/evaluation data as JSON endpoints	<code>pip install flask</code>
flask-cors	>=4.0	Cross-origin requests from React frontend to Python backend	<code>pip install flask-cors</code>
Chart.js	v4 CDN	Bar charts, doughnut charts, line charts for flow/reward visualization	CDN link in HTML
React	18+ CDN	Component-based dashboard UI with persona switching and live updates	CDN or <code>create-react-app</code>

5.4 One-Command Install

```
pip install stable-baselines3 gymnasium numpy pygame matplotlib tensorboard pandas flask  
flask-cors
```

Verify installation:

```
python -c "import stable_baselines3; import gymnasium; import numpy; import flask; print('ALL  
GOOD')"
```

6. Components to Develop

06 · COMPONENTS TO DEVELOP

6.1 Project File Structure

```
flowstate-rl/ environment/ personas.py flow_env.py test_env.py agent/ train.py evaluate.py
models/ game1_shooter/ player.py enemy.py metrics.py game_env.py main.py game2_runner/
runner.py runner_env.py dashboard/ index.html data/ training_results.json
evaluation_results.json
```

6.2 Environment Components

File	Class/Function	Responsibility	Key Methods
<code>personas.py</code>	Beginner, Average, Expert, Erratic, PersonaFactory	Simulate 4 player types as data generators. Return 6D state vectors with realistic noise. No game needed.	<code>step()</code> -> <code>np.array(6)</code> <code>reset()</code> <code>get_name()</code>
<code>flow_env.py</code>	FlowEnv (gym.Env)	Core Gymnasium environment. Accepts persona, applies actions to game params, computes Flow Theory reward.	<code>reset()</code> -> <code>obs, info</code> <code>step(action)</code> -> <code>obs, r, term, trunc, info</code> <code>_compute_reward()</code>
<code>test_env.py</code>	Validation Script	Validates environment correctness before training. Tests obs shape, reward range, episode length.	<code>run_tests()</code> <code>print_summary()</code>

6.3 Agent Components

File	Purpose	Input	Output
<code>train.py</code>	Train one PPO model per persona (4 total). Log to TensorBoard.	FlowEnv + persona name	<code>./models/ppo_flowstate_{persona}.zip</code> <code>./data/training_results.json</code>
<code>evaluate.py</code>	Load trained models, run 10 eval episodes each, compare vs 3 baselines.	Trained .zip models FlowEnv instances	<code>./data/evaluation_results.json</code> Formatted console table

6.4 Game 1 — Top-Down Shooter Components

File	Class	Responsibility	Key Attributes
<code>player.py</code>	Player	WASD movement, click-to-shoot, HP system, invincibility frames, shot tracking	<code>hp</code> , <code>position</code> , <code>shots_fired</code> , <code>deaths</code> , <code>damage_taken</code>
<code>enemy.py</code>	Enemy Enemy Manager	Spawn from edges, chase player via normalized direction vector, configurable stats	<code>speed</code> , <code>hp</code> , <code>spawn_rate</code> (all RL-controlled)
<code>metrics.py</code>	MetricsCollector	Tracks all 6 state features in real-time. Samples every 60 ticks. Returns normalized S.	<code>kill_history</code> , <code>health_log</code> , <code>dodge_tracker</code>

<code>game_env.py</code>	ShooterEnv (gym.Env)	Gymnasium wrapper for the full Pygame game. Connects RL agent to actual game loop.	<code>step()</code> , <code>reset()</code> , <code>render()</code>
<code>main.py</code>	GameLoop	Full game loop at 60 FPS. HUD with HP bar, score, difficulty params, flow status.	<code>human_mode</code> flag, <code>agent_mode</code> flag

6.5 Dashboard Components

Component	Type	Description	Data Source
Flow Zone Gauge	RadialBar	% of timesteps agent kept player in Flow Zone per persona	<code>evaluation_results.json</code> <code>mean_flow_pct</code>
Reward Curve	LineChart	Episode reward over training timesteps, one line per persona	<code>training_results.json</code> <code>mean_reward</code>
Player State Bar	Animated Badge	Live indicator: FLOW (cyan) / OVERWHELMED (red) / BORED (yellow)	Real-time or simulated cycle
Action Heatmap	BarChart	Which of 9 actions PPO agent prefers — reveals learned strategy	<code>evaluation_results.json</code> <code>most_common_action</code>
Persona Tabs	Button Group	Switch between Beginner/Average/Expert/Erratic — updates all charts	React state <code>currentPersona</code>
Eval Comparison	Grouped Bar	PPO vs Random vs Static Easy vs Static Hard — flow%, reward, overwhelm%	<code>evaluation_results.json</code> all policies

7. Development Timeline

07 · 8-WEEK DEVELOPMENT TIMELINE

Week	Phase	Tasks	Deliverable	Files
Week 1	Environment Setup	Build <code>personas.py</code> , <code>flow_env.py</code> , <code>test_env.py</code> . Validate all 4 personas + Gym env.	Working Gym environment	<code>personas.py</code> <code>flow_env.py</code> <code>test_env.py</code>
Week 2	Agent Training	Build <code>train.py</code> . Train PPO on all 4 personas x 50k timesteps each. Monitor TensorBoard.	4 trained PPO models	<code>train.py</code> <code>models/*.zip</code>
Week 3	Evaluation + Dashboard	Build <code>evaluate.py</code> . Compare PPO vs 3 baselines. Build <code>index.html</code> dashboard.	Dashboard with charts	<code>evaluate.py</code> <code>index.html</code>
Week 4	Game 1 Core	Build Player, Enemy, MetricsCollector classes in Pygame. Test standalone game.	Playable game	<code>player.py</code> <code>enemy.py</code> <code>metrics.py</code>
Week 5	Game 1 Integration	Build ShooterEnv Gym wrapper. Connect RL agent to live game. Add HUD overlay.	RL-controlled game	<code>game_env.py</code> <code>main.py</code>
Week 6	Game 1 Training	Retrain PPO agent on real Pygame game. Replace personas with live metrics. Tune rewards.	Game-trained model	<code>train.py</code> <code>updated new models/</code>
Week 7	Game 2 Runner	Build Endless Runner game + RunnerEnv. Test if agent transfers from Game 1.	Transfer learning test	<code>runner.py</code> <code>runner_env.py</code>
Week 8	Human Testing	Real human playtesting sessions. Record flow zone %. Post-session survey. Final report.	Final paper + demo video	<code>report.pdf</code> <code>demo.mp4</code>

8. Dashboard Design Plan

08 · DASHBOARD DESIGN PLAN

The FlowState-RL dashboard is a single HTML file using Chart.js. It loads evaluation results from a JSON file and presents all metrics visually. The dashboard works with both real training data and fallback mock data for demonstration purposes.

Section	Content	Chart Type
Header	Title, subtitle, animated LIVE badge	CSS animation
Stat Cards	Best Flow%, Best Reward, Best Persona, PPO vs Random improvement	Count-up animation
Persona Tabs	4 buttons: Beginner / Average / Expert / Erratic	React state
Row 1 Charts	Flow Zone % bar + Mean Reward bar (4 policies each)	Chart.js Bar

Row 2 Charts	Player State doughnut + All Personas Flow % horizontal bar	Chart.js Doughnut + Bar
Comparison Table	Policy Flow% Overwhelmed% Bored% Reward	HTML Table + CSS
Footer	Tech stack credits	Static text

Data Flow: Python -> JSON -> Dashboard

```
evaluate.py --> ./data/evaluation_results.json --> fetch() in index.html --> Chart.js renders
```

```
train.py --> ./data/training_results.json --> fetch() in index.html --> reward curve chart
```

```
main.py (game) --> ./data/live_metrics.json --> server.py (Flask) --> localhost:5000/metrics
--> dashboard auto-fetch
```

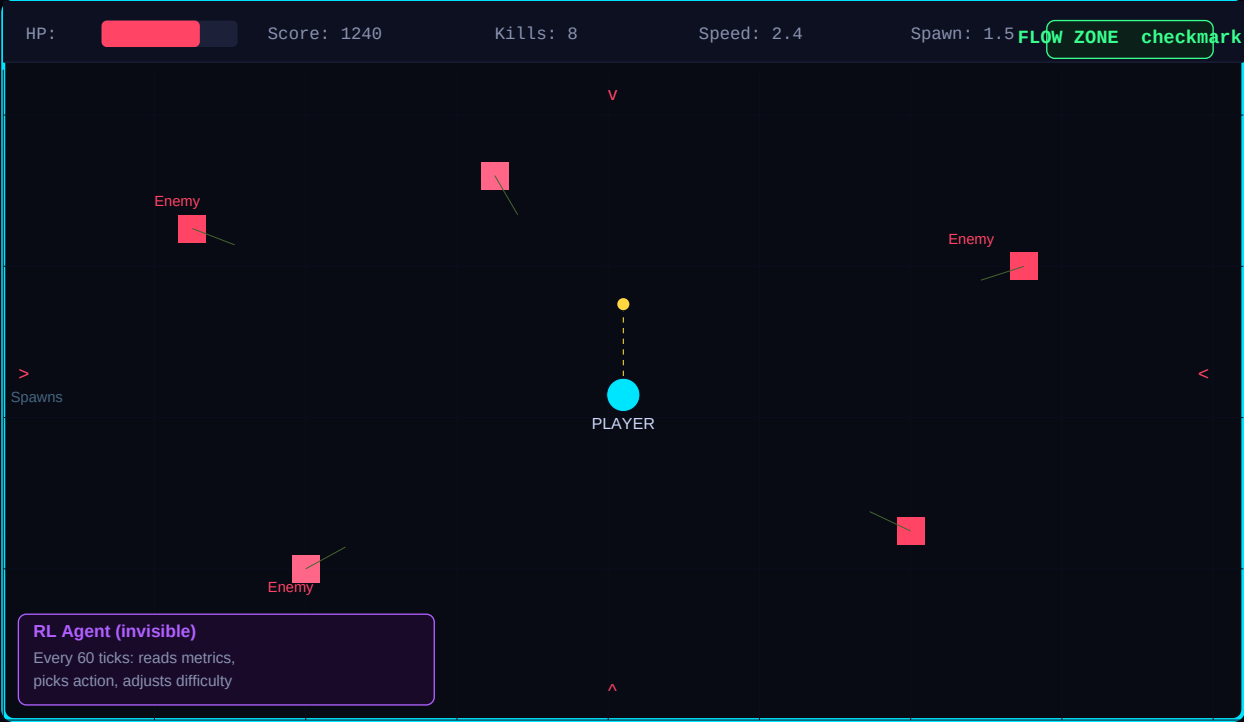
9. How to Play the Game

09 · GAME CONTROLS & WINDOW LAYOUT

9.1 Player Controls

Input	Action	Input	Action
W	Move Up	Mouse Move	Aim bullet direction
A	Move Left	Left Click	Shoot bullet toward cursor
S	Move Down	ESC	Pause / Quit game
D	Move Right	R	Restart episode

9.2 Game Window Layout (800 x 600 px)



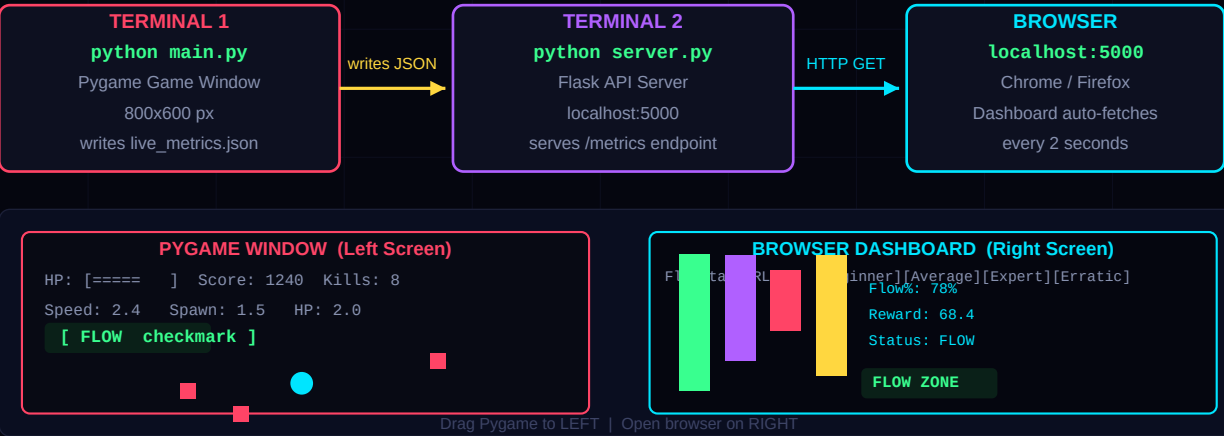
9.3 HUD Elements Explained

HUD Element	What It Shows	Updates
HP Bar (red/green)	Player current health out of 100. Red when low.	Every frame
Score Counter	Total points earned from kills and survival time	Every kill
Kill Count	Enemies eliminated this episode	Every kill
Speed / Spawn / HP	Current RL agent difficulty parameters	Every 60 ticks
FLOW / BORED / OVERWHELMED badge	Current player state as computed by agent	Every 60 ticks

10. Dual-Screen Demo Setup

10 · RUNNING GAME + DASHBOARD SIDE BY SIDE

10.1 Architecture: 3 Processes Running Together



10.2 Step-by-Step Launch Instructions

Step	Command	What Happens
1	<code>cd flowstate-rl</code>	Navigate to project folder in CMD
2	<code>python train.py</code>	Train PPO models (run once, takes ~10 min). Saves to <code>./models/</code>
3	<code>python evaluate.py</code>	Generate <code>evaluation_results.json</code> for dashboard charts
4	<code>python server.py</code> (Terminal 2)	Start Flask server at <code>localhost:5000</code> . Keep this running.
5	<code>python main.py</code> (Terminal 1)	Opens Pygame game window. Drag to LEFT side of screen.
6	Open Chrome -> <code>localhost:5000</code>	Dashboard loads. Drag browser to RIGHT side. Play the game!

10.3 Live Data Update Flow

<code>main.py</code>	---writes-->	<code>./data/live_metrics.json</code> (every 60 game ticks)
<code>server.py</code>	--reads-->	<code>live_metrics.json</code> --serves--> GET <code>/metrics</code> (Flask endpoint)
<code>index.html</code>	--fetches-->	<code>localhost:5000/metrics</code> --updates--> all charts (every 2 seconds)

Dashboard Element	Updates Every	Source	JSON Key
FLOW / BORED / OVERWHELMED badge	2 seconds	live game state	<code>current_state</code>
Enemy Speed display	2 seconds	agent action	<code>enemy_speed</code>
Spawn Rate display	2 seconds	agent action	<code>spawn_rate</code>
Flow Zone % gauge	2 seconds	metrics history	<code>flow_pct</code>
Score & Kill Count	2 seconds	game state	<code>score, kills</code>

11. server.py — Flask Bridge Component

11 · FLASK SERVER: GAME TO DASHBOARD BRIDGE

server.py is the critical bridge component that connects the live Pygame game to the browser dashboard. It runs as a separate Flask process, reads the live_metrics.json file written by the game every 60 ticks, and serves it via HTTP so the dashboard can fetch and display real-time metrics.

11.1 server.py Responsibilities

Endpoint	Method	Returns	Used By
/	GET	Serves index.html dashboard file	Browser on load
/metrics	GET	live_metrics.json (current game state)	Dashboard every 2s
/evaluation	GET	evaluation_results.json (trained model results)	Dashboard on load
/training	GET	training_results.json (PPO training history)	Dashboard on load
/status	GET	Server health check: {"status": "running"}	Dashboard ping

11.2 live_metrics.json Structure

```
{
  "kill_rate": 0.52,
  "death_rate": 0.18,
  "avg_health": 0.74,
  "dodge_success": 0.61,
  "score_velocity": 0.55,
  "danger_time": 0.22,
  "score": 1240,
  "kills": 8,
  "enemy_speed": 2.4,
  "spawn_rate": 1.5,
  "enemy_hp": 2.0,
  "current_state": "FLOW",
  "flow_pct": 78,
  "episode_reward": 145.5,
  "last_action": 2,
  "flow_streak": 42,
  "timestamp": 1709912345
}
```

12. Complete Development Prompts

12 · ALL 7 CODE GENERATION PROMPTS

Use these prompts with any AI coding assistant (Claude, ChatGPT, Copilot) to generate each file. Paste each prompt exactly as written. Generate one file at a time, test it, then move to the next prompt.

Prompt 1 — personas.py

Create personas.py for FlowState-RL with 4 player classes: Beginner, Average, Expert, Erratic. Each has step() returning np.array(6,) of [kill_rate, death_rate, avg_health, dodge_success, score_velocity, danger_time] normalized to [0,1]. Beginner: low kills(0.15), high deaths(0.75). Expert: high kills(0.85), low deaths(0.08). Average: moderate all(0.5). Erratic: switches between Expert/Beginner every 20 steps. Add PersonaFactory.get_persona(name) static method. Include __main__ test block printing state table for all 4 personas.

Run: python personas.py

Prompt 2 — flow_env.py

Create FlowEnv(gym.Env) in flow_env.py. observation_space=Box(6,) float32. action_space=Discrete(9). Actions: 0-2 enemy_speed +/-0.2, 3-5 spawn_rate +/-0.2, 6-8 enemy_hp +/-0.5. Reward: +2.0 flow zone(perf 0.5-0.7), +0.5 sustained>30 steps, -1.0 overwhelmed(death>0.7), -0.5 bored(kill>0.8 and death<0.1), -0.1 otherwise. 200 step episodes. Accept persona_name in __init__. Full Gymnasium API: reset(), step() with terminated/truncated.

Run: `python flow_env.py`

Prompt 3 — test_env.py

Create test_env.py validating FlowEnv with all 4 personas. Run 5 episodes x 200 steps per persona using random policy. Assert obs shape==(6,), all values in [0,1], rewards in {-1.0,-0.5,-0.1,+2.0,+2.5}, truncated at step 200. Print summary table: persona | mean_reward | flow% | overwhelmed% | bored%. Print PASS/FAIL for each test.

Run: `python test_env.py`

Prompt 4 — train.py

Create train.py using stable-baselines3 PPO. For each of 4 personas train: MlpPolicy, lr=3e-4, n_steps=2048, batch=64, epochs=10, gamma=0.99, clip=0.2, ent_coef=0.01, net_arch=[128,64], 50000 timesteps. Save to ./models/ppo_flowstate_{persona}. Evaluate with evaluate_policy(n_eval=10). Save training_results.json. Log to tensorboard ./logs/. Print formatted summary table.

Run: `python train.py`

Prompt 5 — evaluate.py

Create evaluate.py comparing PPO vs 3 baselines. Load each ./models/ppo_flowstate_{persona}. Run 10 episodes each policy: PPO(deterministic), Random, StaticEasy(actions 0,3,6), StaticHard(actions 2,5,8). Track: mean_reward, flow_pct%, overwhelmed_pct%, bored_pct%, most_common_action. Save evaluation_results.json. Print comparison table per persona.

Run: `python evaluate.py`

Prompt 6 — server.py

Create server.py Flask API server. Endpoints: GET / serves dashboard/index.html, GET /metrics reads and returns ./data/live_metrics.json, GET /evaluation returns ./data/evaluation_results.json, GET /training returns ./data/training_results.json, GET /status returns running status. Enable CORS with flask-cors. Run on port 5000. If JSON file missing return empty dict {}. Add if __main__ block to start server.

Run: `python server.py`

Prompt 7 — dashboard/index.html

Build single-file HTML dashboard for FlowState-RL. Load /metrics every 2 seconds via fetch(). Load /evaluation on page load. Show: (1) Flow Zone % RadialBar gauge, (2) FLOW/BORED/OVERWHELMED animated status badge, (3) current difficulty params (speed/spawn/hp), (4) Flow% bar chart comparing PPO vs Random vs Static Easy vs Static Hard for all 4 personas, (5) Doughnut chart showing flow/overwhelmed/bored% split, (6) persona selector tabs, (7) comparison table. Dark theme #05060f, cyan #00e5ff. Use Chart.js from CDN.

Run: open in browser

13. Complete File Reference

13 · ALL FILES, CLASSES AND RUN COMMANDS

File	Layer	Key Classes	Run Command	Output
<code>personas.py</code>	Environment	Beginner, Average, Expert, Erratic, PersonaFactory	<code>python personas.py</code>	Console state table
<code>flow_env.py</code>	Environment	FlowEnv(gym.Env)	<code>python flow_env.py</code>	Episode test output
<code>test_env.py</code>	Validation	<code>run_tests()</code> , <code>print_summary()</code>	<code>python test_env.py</code>	PASS/FAIL report
<code>train.py</code>	Agent	PPO (stable-baselines3)	<code>python train.py</code>	<code>./models/*.zip</code> + <code>training_results.json</code>
<code>evaluate.py</code>	Agent	<code>run_evaluation()</code> , baseline policies	<code>python evaluate.py</code>	<code>evaluation_results.json</code>
<code>player.py</code>	Game 1	Player	(imported by <code>main.py</code>)	Player sprite + controls
<code>enemy.py</code>	Game 1	Enemy, EnemyManager	(imported by <code>main.py</code>)	Enemy spawning + AI
<code>metrics.py</code>	Game 1	MetricsCollector	(imported by <code>main.py</code>)	6D state vector
<code>game_env.py</code>	Game 1	ShooterEnv(gym.Env)	(imported by <code>train.py</code>)	Gym wrapper
<code>main.py</code>	Game 1	GameLoop	<code>python main.py</code>	Pygame window 800x600
<code>server.py</code>	Bridge	Flask app, <code>/metrics</code> , <code>/evaluation</code>	<code>python server.py</code>	<code>localhost:5000</code> API
<code>dashboard/index.html</code>	Dashboard	Chart.js components, persona tabs	<code>localhost:5000</code>	Browser dashboard

Quick-Start Checklist

[] Install libraries	<code>pip install stable-baselines3 gymnasium numpy pygame matplotlib flask flask-cors</code>
[] Verify install	<code>python -c "import stable_baselines3; import gymnasium; print('ALL GOOD')"</code>
[] Build environment	<code>python personas.py</code> then <code>python flow_env.py</code> then <code>python test_env.py</code>
[] Train agent	<code>python train.py</code> (creates <code>./models/</code> folder with 4 PPO models)
[] Evaluate	<code>python evaluate.py</code> (creates <code>evaluation_results.json</code>)

-
- | | |
|--------------------|--|
| [] Start server | python server.py (keep running in Terminal 2) |
| [] Play the game | python main.py (drag to left half of screen) |
| [] View dashboard | Open Chrome -> localhost:5000 (drag to right half of screen) |

References

Csikszentmihalyi, M. (1990). Flow: The Psychology of Optimal Experience. Harper & Row.
Schulman, J. et al. (2017). Proximal Policy Optimization Algorithms. arXiv:1707.06347.
Brockman, G. et al. (2016). OpenAI Gym. arXiv:1606.01540.
Raffe, W.L. et al. (2012). A Survey of Procedural Terrain Generation. IEEE CIG 2012.